CprE 381, Computer Organization and Assembly-Level Programming

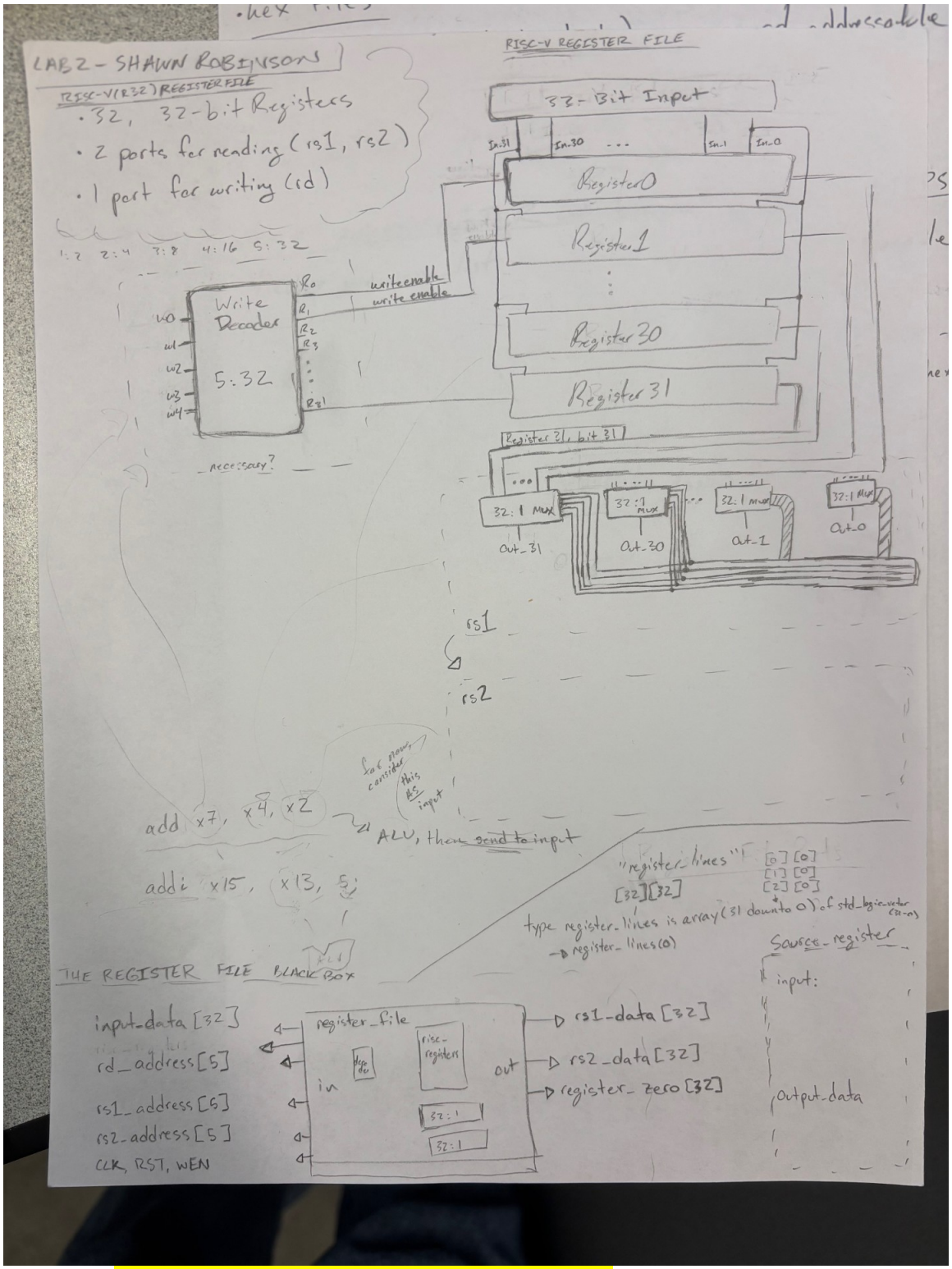
Lab 2 Report

Student Name SHAWN ROBINSON

PRELAB: The D Flip Flop Gate that is provided takes in four inputs and has one output. All of these are single-wire std_logic lines. There is a “with...select” structure used to set the data line like a multiplexer...Also, based off the reading it looks like the process implicitly creates stored values by not having a conclusive else statement in the if checks of the lower down process...it only checks for rising edges of i_CLK and if i_RST is 1.

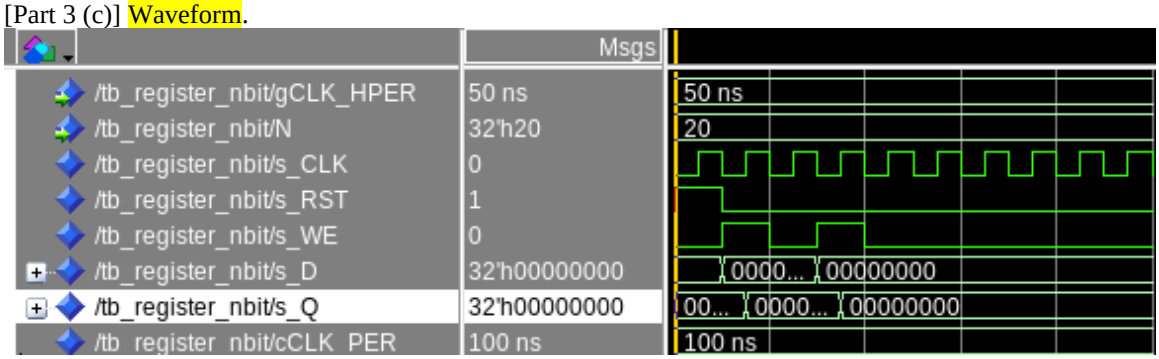
Submit a typeset pdf version of this on Canvas by the due date. Refer to the highlighted language in the lab document for the context of the following questions.

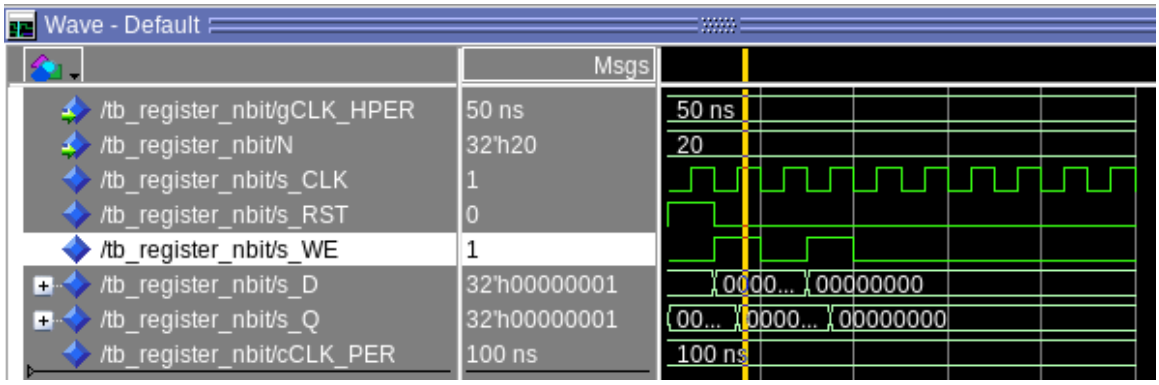
[Part 3 (a)] Draw the interface description (i.e., the “symbol” or high-level blackbox) for the RISC-V register file. Which ports do you think are necessary, and how wide (in bits) do they need to be?

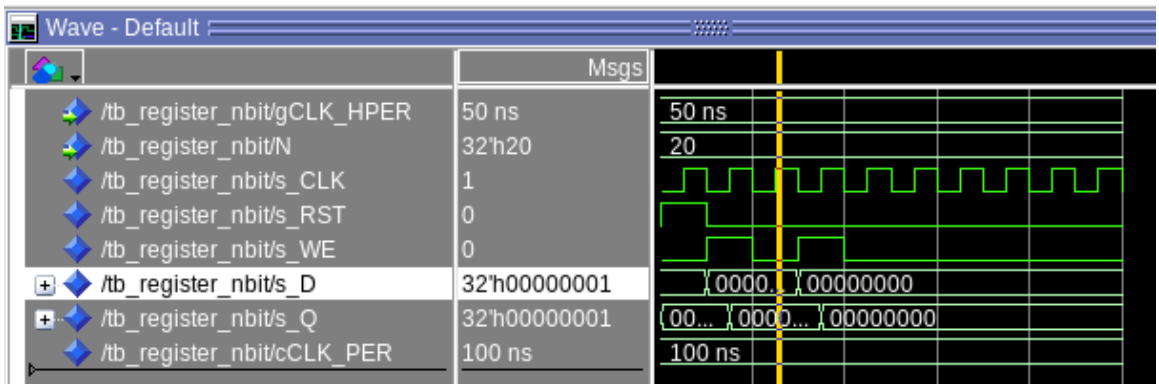


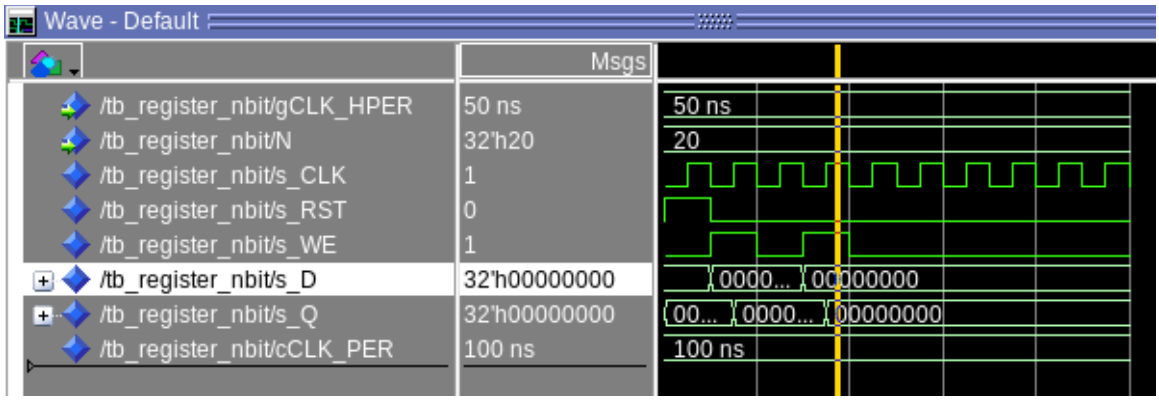
[Part 3 (b)] Create an N-bit register using this flip-flop as your basis.

[Part 3 (c)] Waveform.



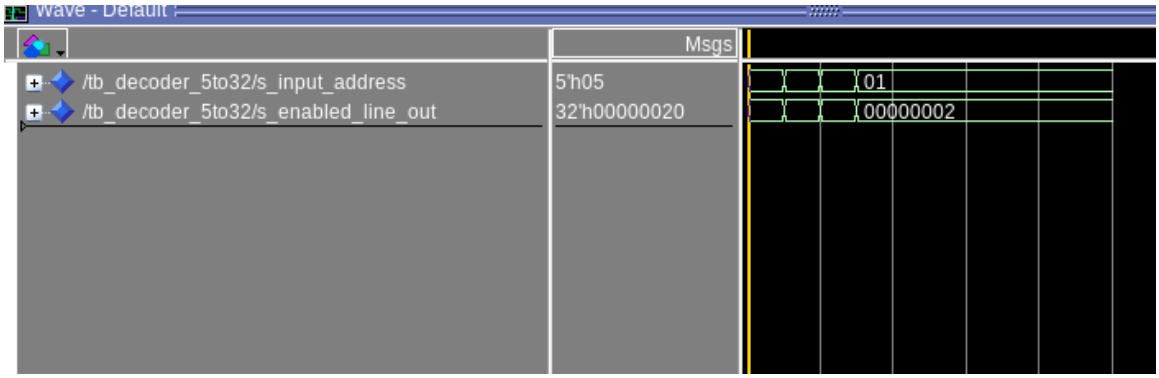


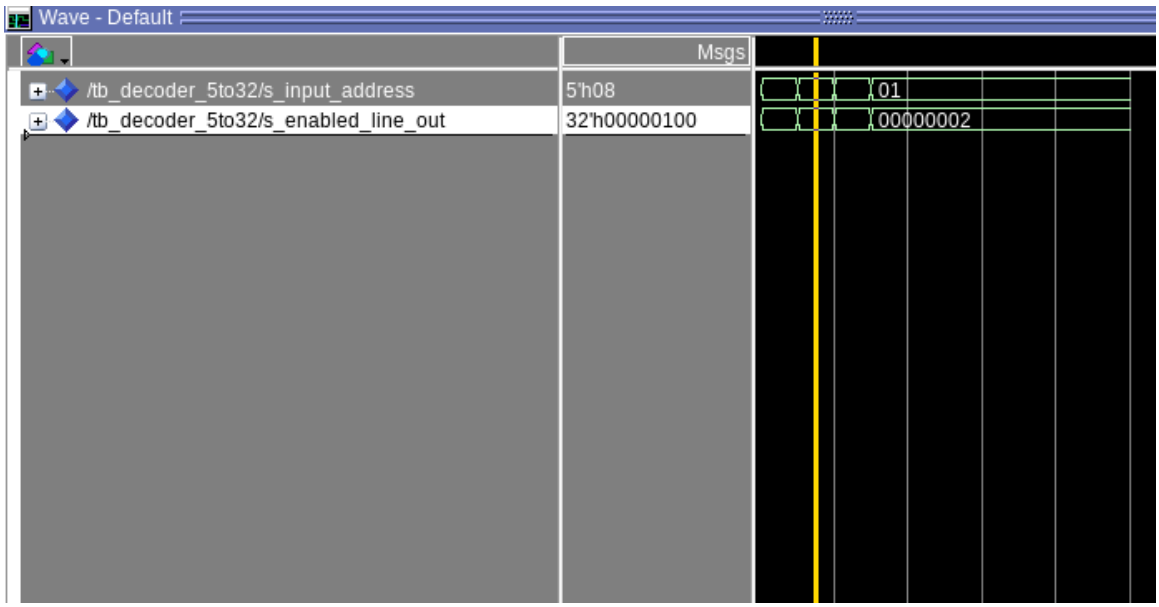


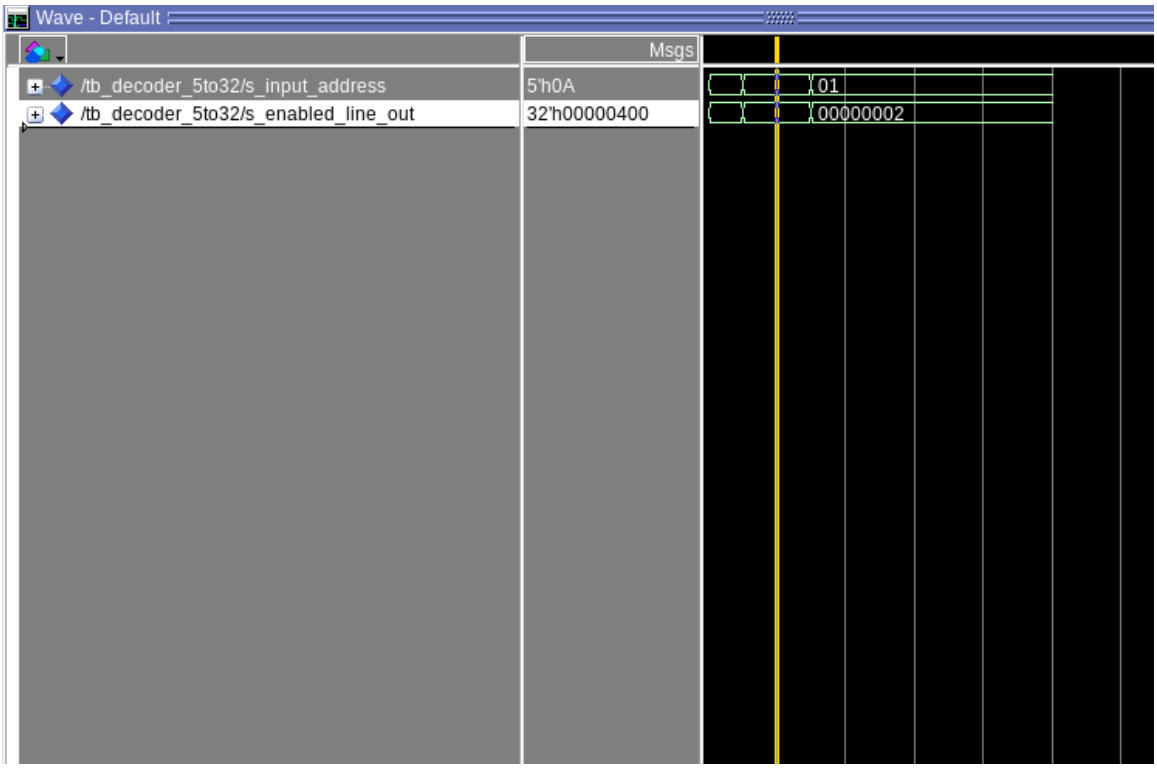


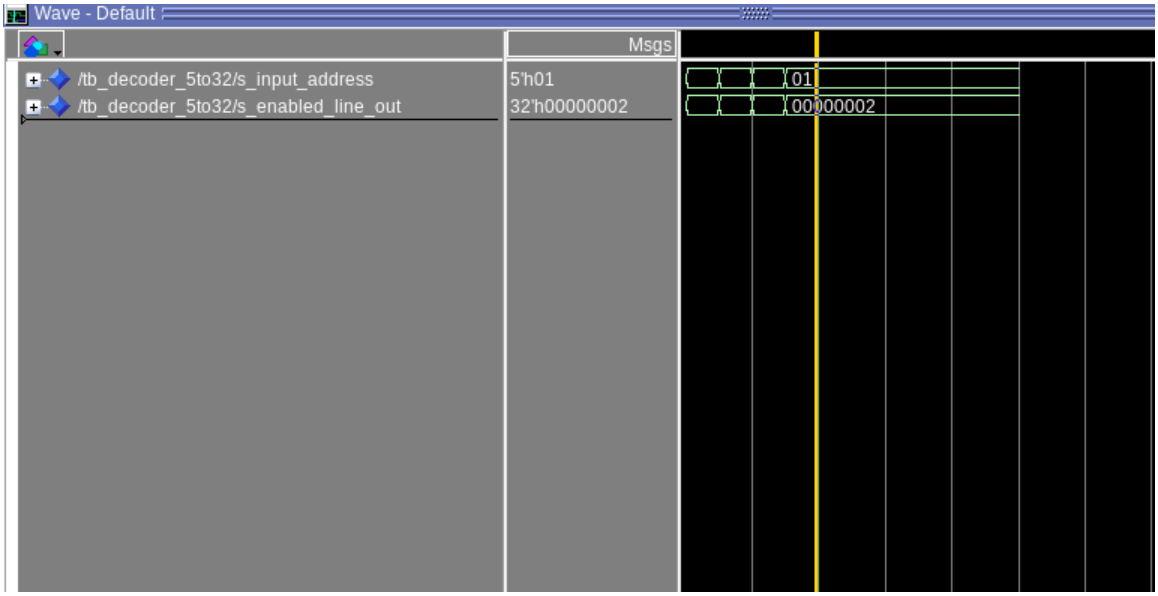
[Part 3 (d)] What type of decoder would be required by the RISC-V register file and why?
The decoder required by the RISC-V register file is a 5 to 32 decoder, because there are 32 registers in the file that can be accessed.

[Part 3 (e)] Waveform.



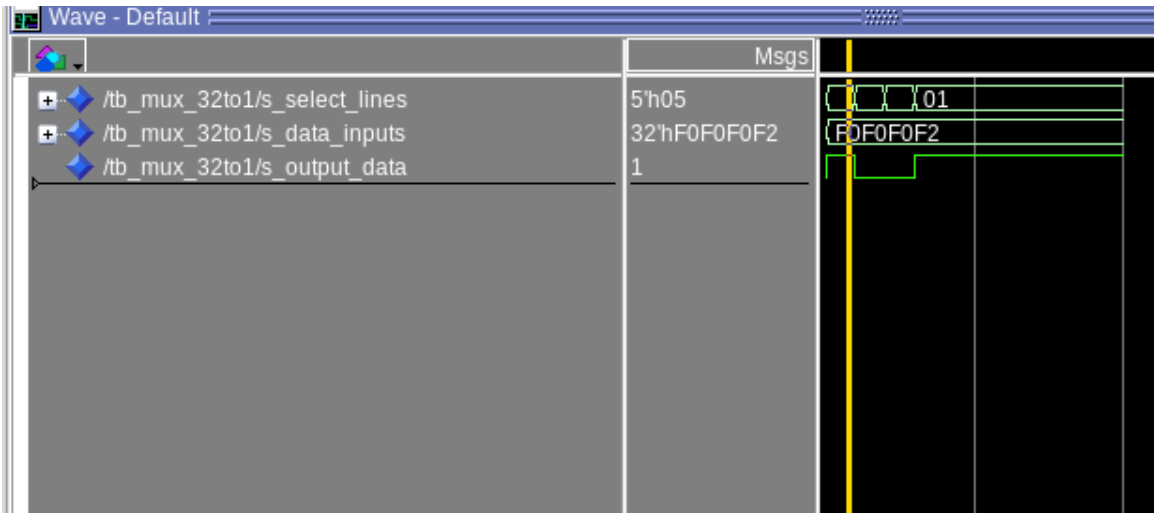


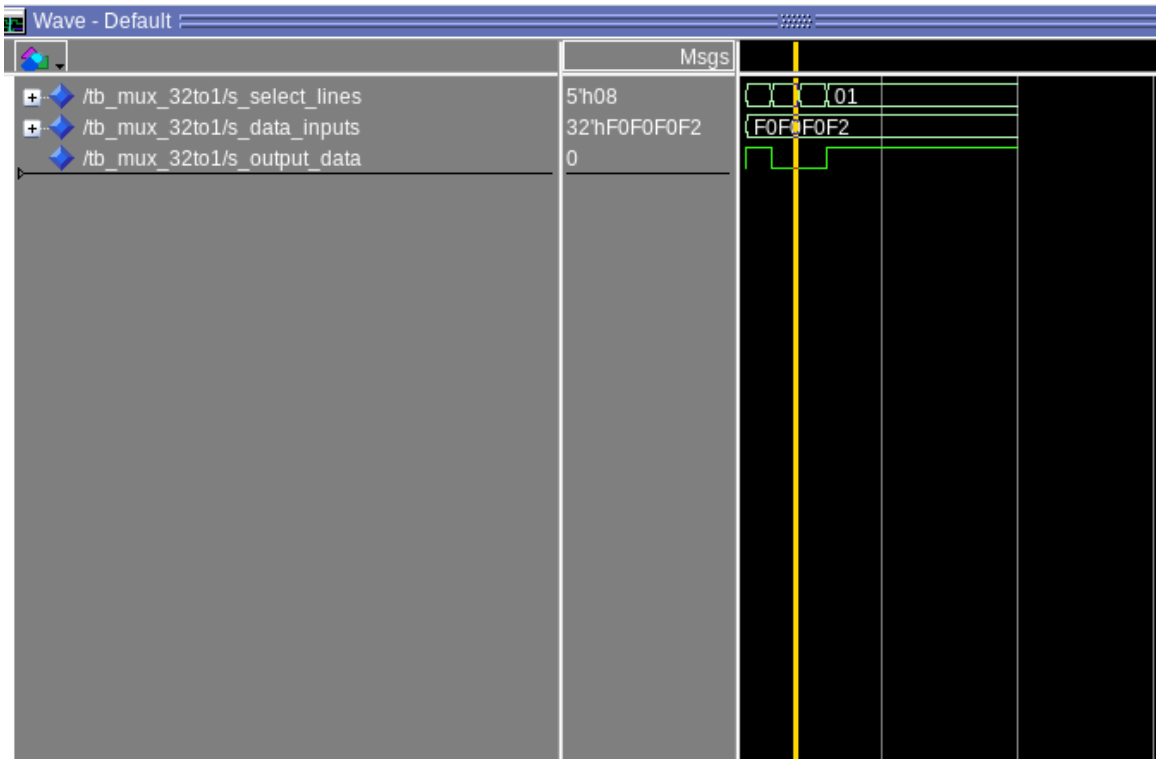


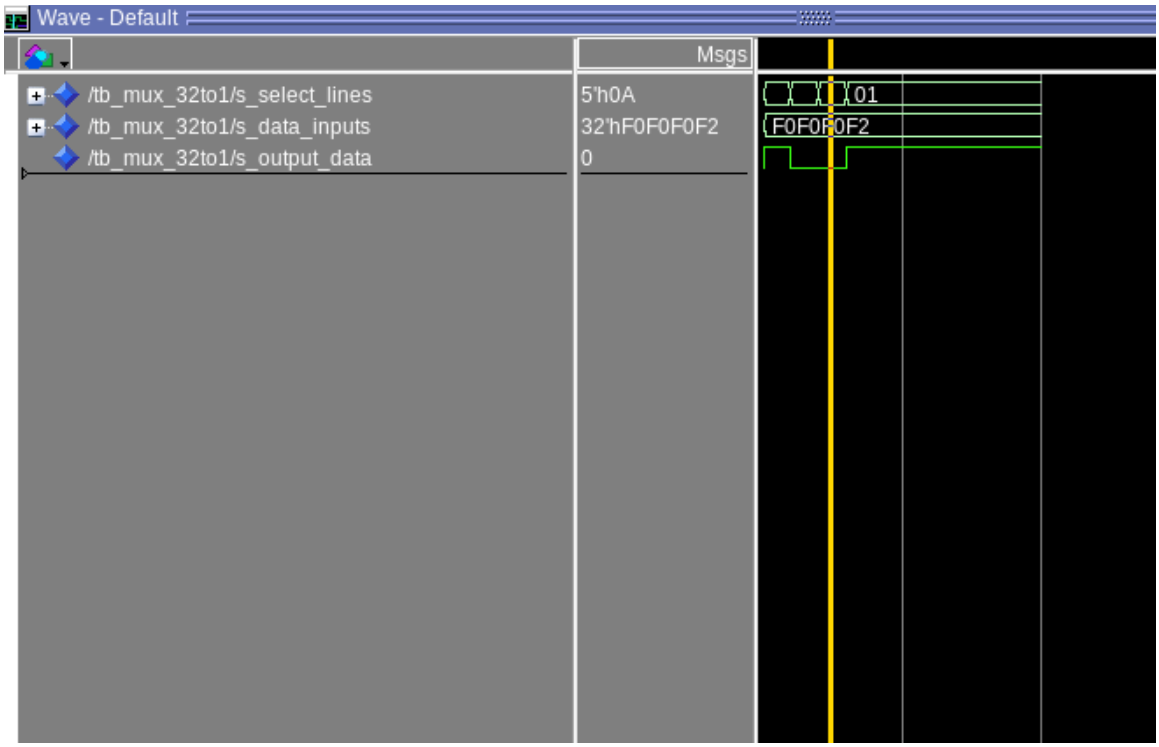


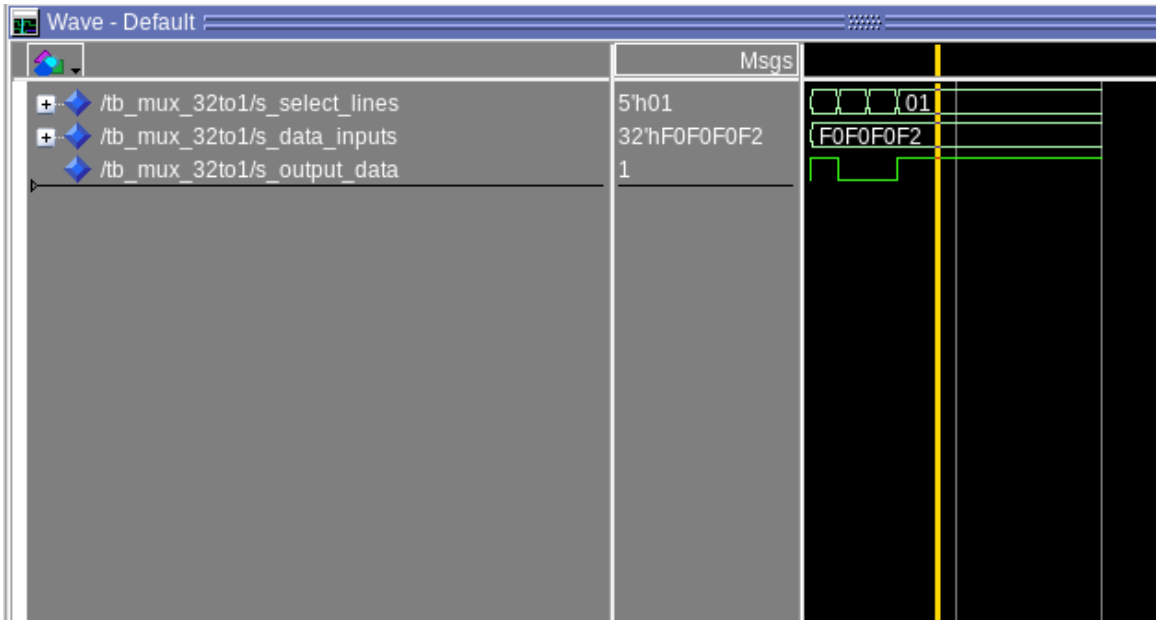
[Part 3 (f)] In your write-up, describe and defend the design you intend on implementing for the next part.
I intend to implement a dataflow design instead of a structural design for the 32:1 MUX. The reason for this is both the ease of initial writing and also the ease of debugging. If we went with a structural system (either with smaller muxes or pure logic gates) there's a lot a more opportunities for an intermediary signal to get misplaced. The dataflow version essentially lets me implement the truth table, which is way easier to check for errors.

[Part 3 (g)] Waveform.






Wave - Default
Msgs
/tb_mux_32to1/s_select_lines
/tb_mux_32to1/s_data_inputs
/tb_mux_32to1/s_output_data
5'h0A
32'hF0F0F0F2
0
01
F0F0F0F2


Wave - Default
Msgs
/tb_mux_32to1/s_select_lines
/tb_mux_32to1/s_data_inputs
/tb_mux_32to1/s_output_data
5'h01
32'hF0F0F0F2
1
01
F0F0F0F2

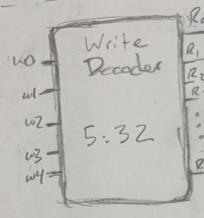
[Part 3 (h)] Draw a (simplified) schematic (i.e., components within the high-level blackbox) for the RISC-V register file, using the same top-level interface ports as in your solution describe above and using only the register, decoder, and mux VHDL components you have created.

LAB 2 - SHAWN ROBINSONS

RISC-V (R32) REGISTER FILE

- 32, 32-bit Registers
- 2 ports for reading (rs1, rs2)
- 1 port for writing (rd)

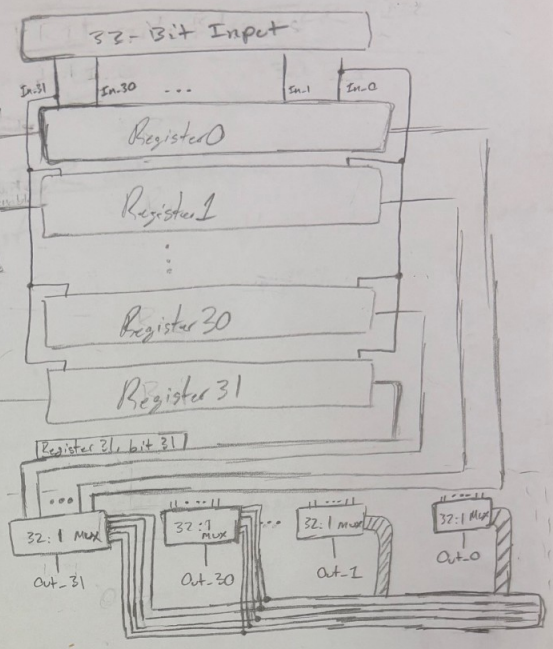
1:2 2:4 3:8 4:16 5:32



writeenable
write enable

necessary?

RISC-V REGISTER FILE



rs1

rs2

for now, consider this as input

add x7, x4, x2 → ALU, then send to input

addi x15, x13, 5;

"register_lines" [32][32]

[0] [0]
[1] [0]
[2] [0]

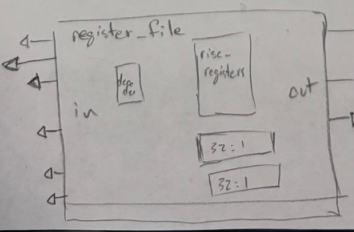
type register_lines is array(31 downto 0) of std_logic_vector(32-0)
→ register_lines(0)

Source_register

input:

output_data

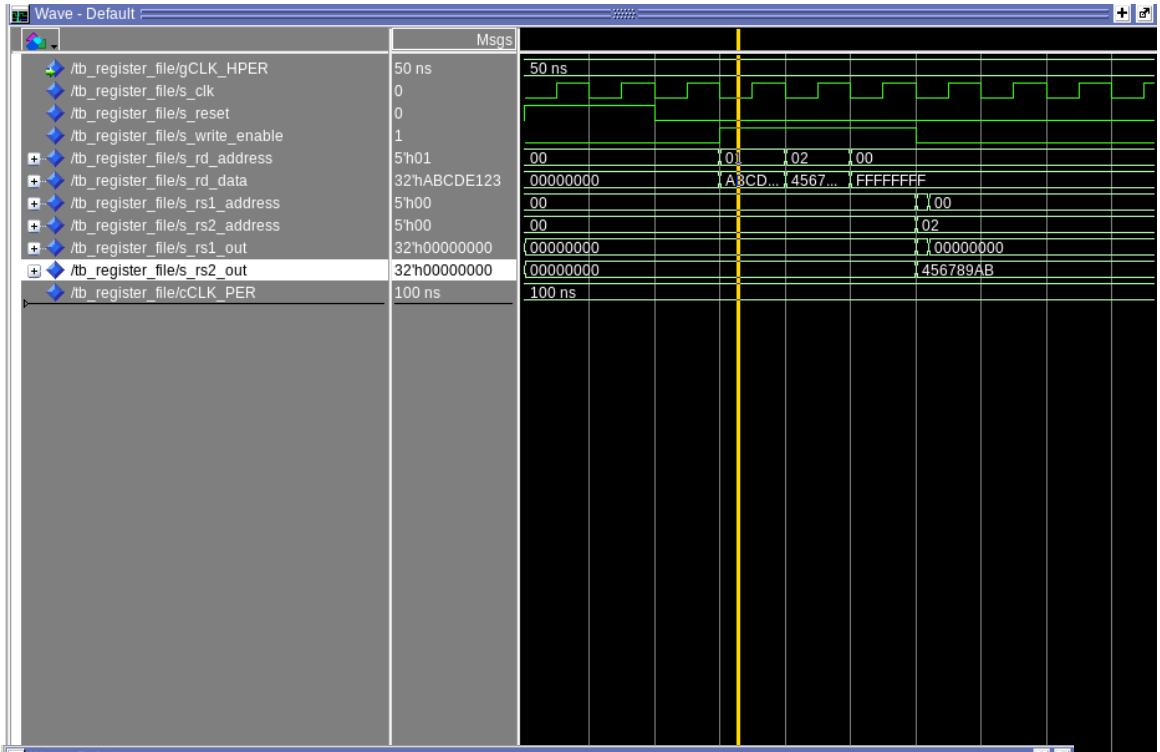
THE REGISTER FILE BLACK BOX

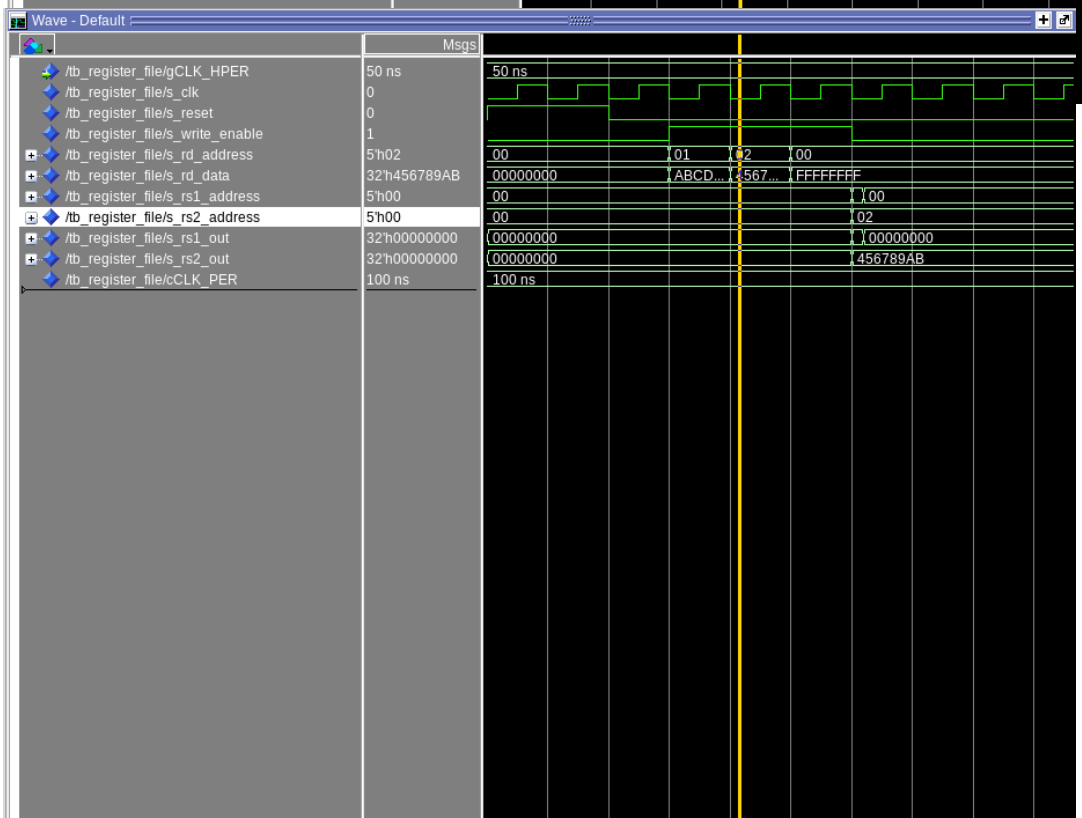


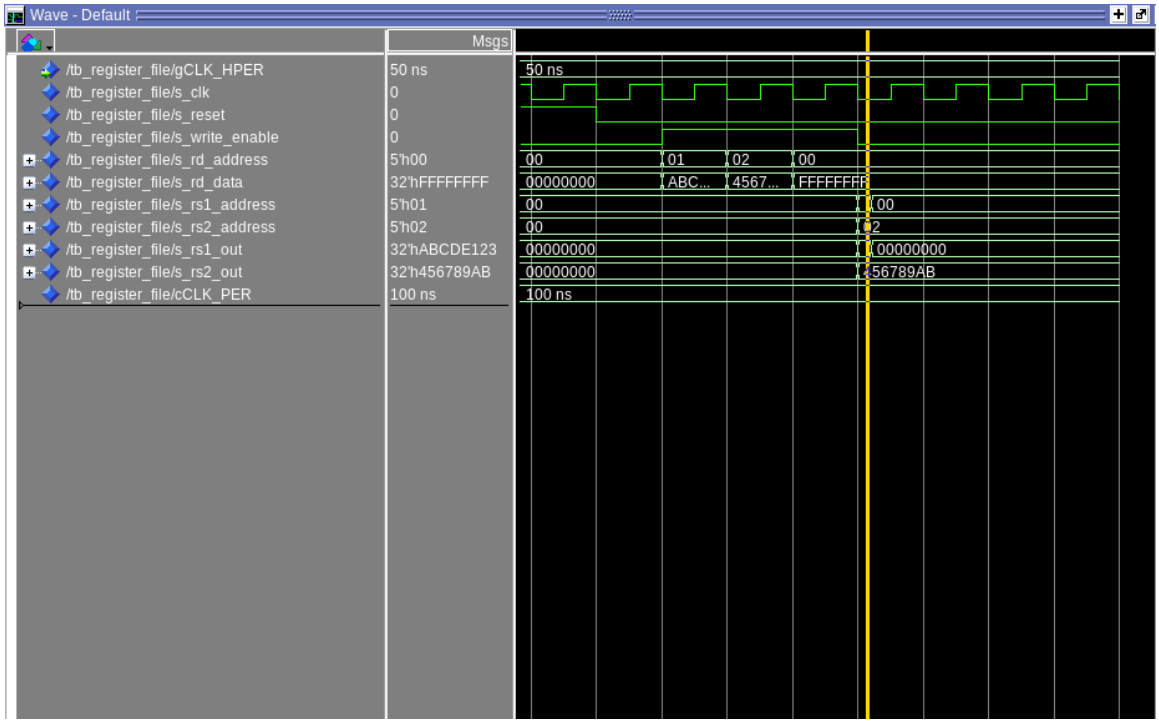
input_data [32]
rd_address [5]
rs1_address [5]
rs2_address [5]
CLK, RST, WEN

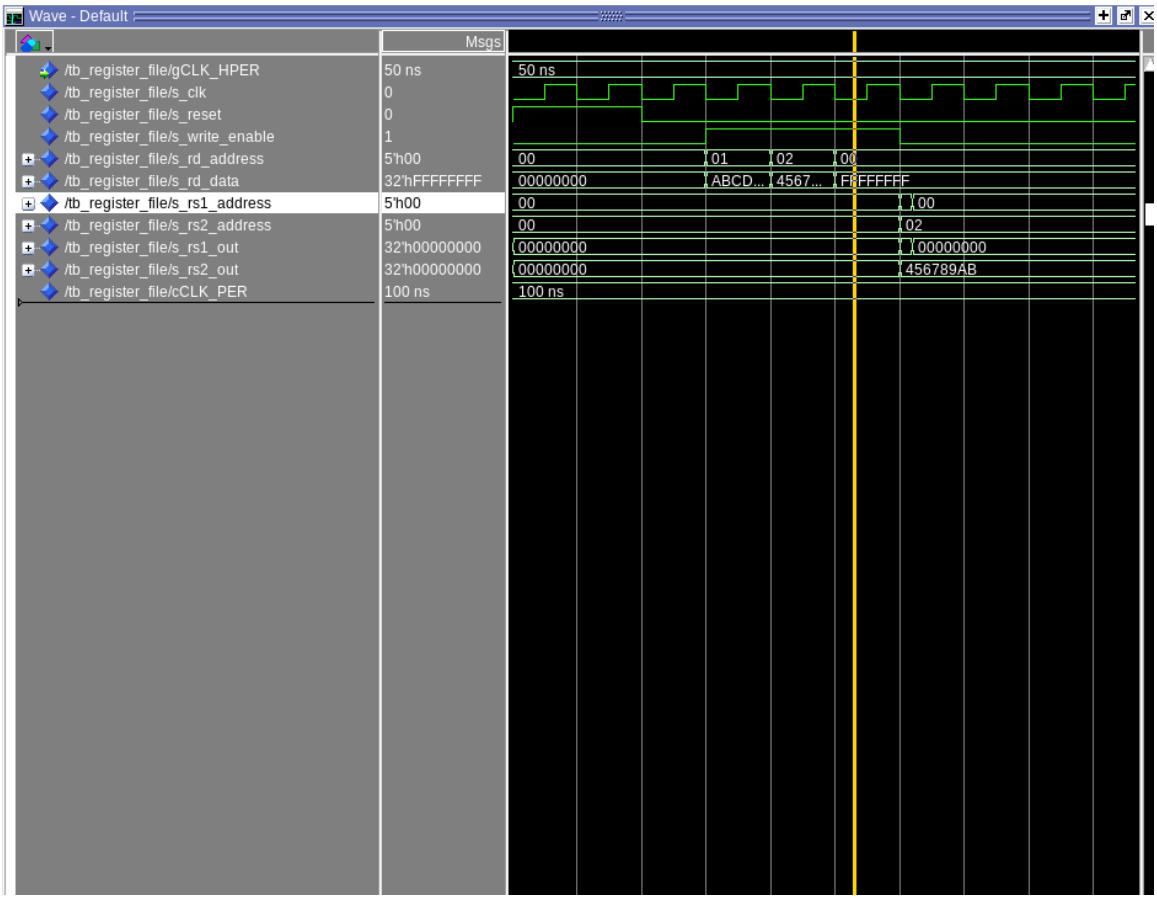
rs1_data [32]
rs2_data [32]
register_zero [32]

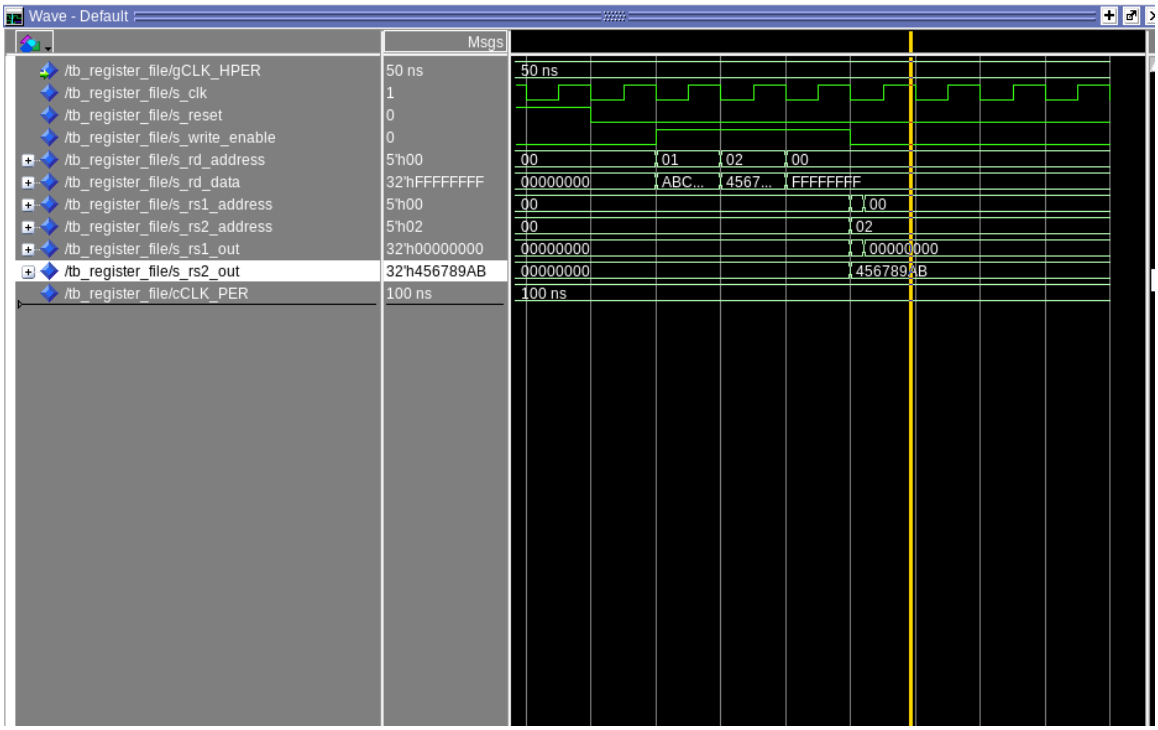
[Part 3 (i)] Waveform.






Wave - Default
Msgs
/tb_register_file/gCLK_HPER 50 ns
/tb_register_file/s_clk 0
/tb_register_file/s_reset 0
/tb_register_file/s_write_enable 0
/tb_register_file/s_rd_address 5'h00
/tb_register_file/s_rd_data 32'hFFFFFFFF
/tb_register_file/s_rs1_address 5'h01
/tb_register_file/s_rs2_address 5'h02
/tb_register_file/s_rs1_out 32'hABCDE123
/tb_register_file/s_rs2_out 32'h456789AB
/tb_register_file/cCLK_PER 100 ns
50 ns
00 01 02 00
00000000 ABC... 4567... FFFFFFFF
00 00
00 02
00000000 00000000
00000000 56789AB
100 ns


Wave - Default
Msgs
/tb_register_file/gCLK_HPER 50 ns
/tb_register_file/s_clk 0
/tb_register_file/s_reset 0
/tb_register_file/s_write_enable 1
/tb_register_file/s_rd_address 5'h00
/tb_register_file/s_rd_data 32'hFFFFFFFF
/tb_register_file/s_rs1_address 5'h00
/tb_register_file/s_rs2_address 5'h00
/tb_register_file/s_rs1_out 32'h00000000
/tb_register_file/s_rs2_out 32'h00000000
/tb_register_file/cCLK_PER 100 ns
50 ns
00
01
02
00
00000000
ABCD...
4567...
FFFFFFFF
00
00
00
02
00000000
00000000
00000000
456789AB
100 ns


Wave - Default
Msgs
/tb_register_file/gCLK_HPER 50 ns
/tb_register_file/s_clk 1
/tb_register_file/s_reset 0
/tb_register_file/s_write_enable 0
/tb_register_file/s_rd_address 5'h00
/tb_register_file/s_rd_data 32'hFFFFFFFF
/tb_register_file/s_rs1_address 5'h00
/tb_register_file/s_rs2_address 5'h02
/tb_register_file/s_rs1_out 32'h00000000
/tb_register_file/s_rs2_out 32'h456789AB
/tb_register_file/cCLK_PER 100 ns
50 ns
00
01
02
00
00000000
ABC...
4567...
FFFFFFFF
00
00
00
02
00000000
00000000
00000000
456789AB
100 ns

[Part 4 (b)] Draw a symbol for this RISC-V-like datapath.

LAB 2 – SHAWN ROBINSON – MORE DATAPATH STUFF

12 bit or 32 for the ALU?

immediate_value

mux2to1

Adder_SubtractorN

register_file

first_datapath

port(CLK in: time;

immediate_value : in std_vector(11 downto 0)

ALUsrc : in std_logic;

reset

write_enable_dp

rs1_address_dp in

rs2_address_dp in

rs1_read_dp out

rs2_read_dp out

rd_address_dp : in

cout → aper

Components

- register_file
- Adder_SubtractorN
- mux2to1_N

Signals

s_rs1_address_dp : std_logic_vector (4 downto 0)

s_result

s_rs1_read_dp

s_rs2_read_dp

s_B_SRC_Choice

s_add_sub_select

s_clk

s_immediate_value

s_ALUsrc

s_reset

s_rs1_address_dp

s_rs2_address_dp

s_rd_address_dp

s_write_enable_dp

begin

register_file

map(clk <= s_clk

reset <= s_reset

write_enable <=

rd_address <= s_rd_address_dp

rd_data <= s_result

rs1_address <= s_rs1_address_dp

rs2_address <= s_rs2_address_dp

[rs1_out <=

rs2_out <]

→ instantiate as 32

Adder_SubtractorN

map(A <= s_rs2_read

B <= s_B_SRC_Choice

add_sub_select <= s_add_sub_select

[result]

datapath

clk <= s_clk

immediate_value <= s_immediate_value

ALUsrc <= s_ALUsrc

reset <= s_reset

rs2_read <= rs2_out

rs2_read <= rs2_out

write_enable_dp <= s_write_enable_dp

instantiate to 32

mux2to1_N

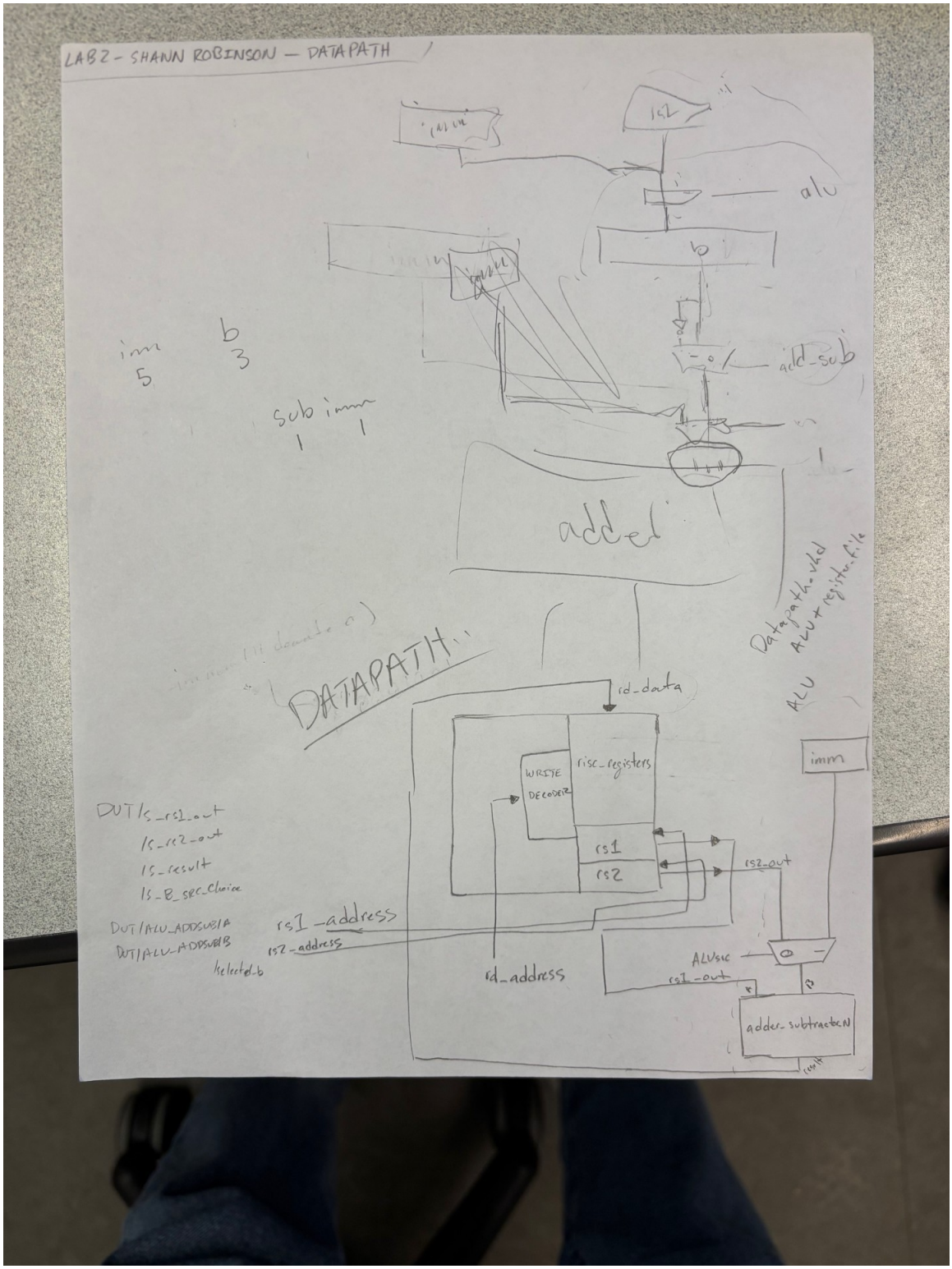
i_S => s_ALUsrc

i_D0 => s_immediate_value

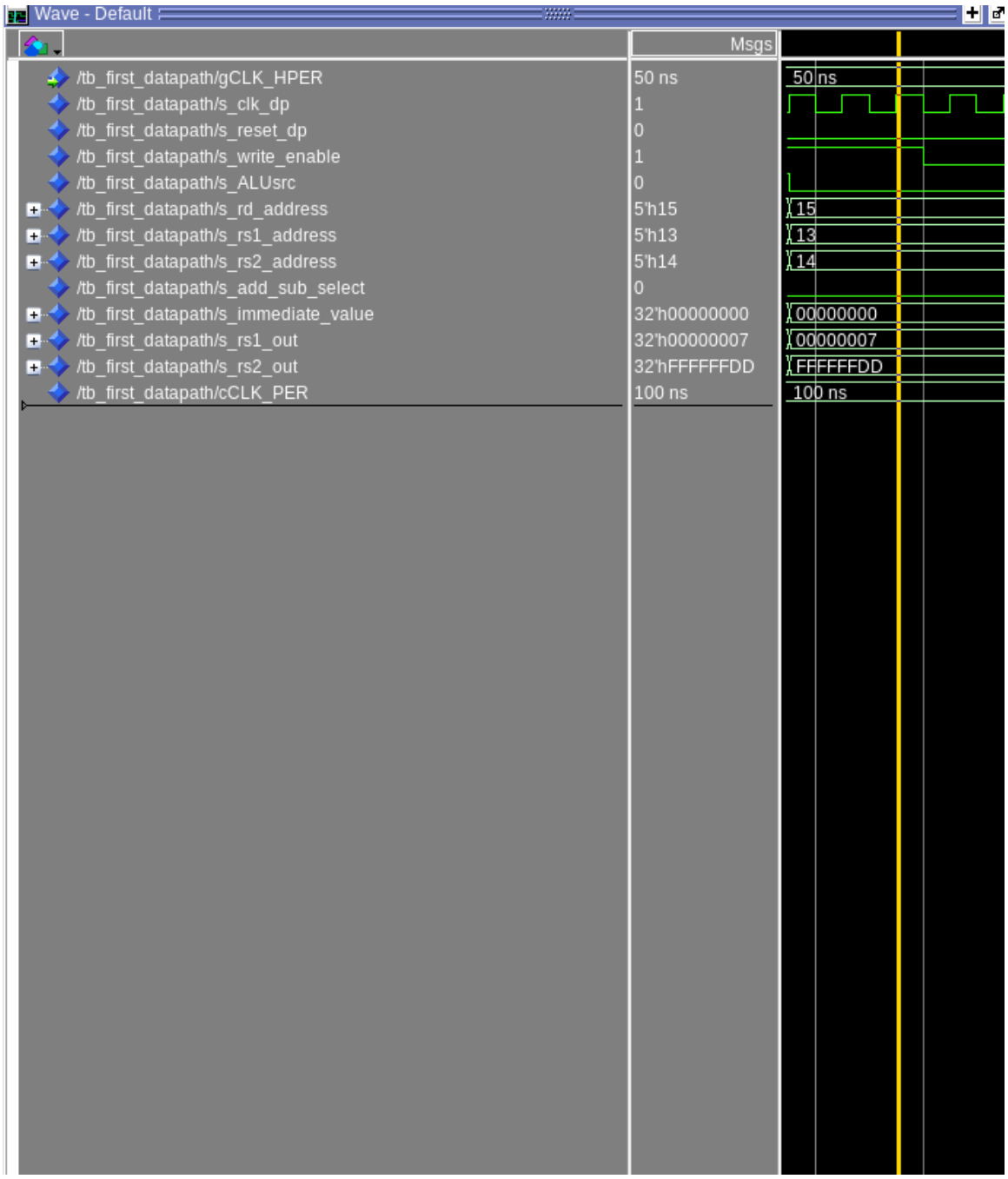
i_D1 => s_rs2_read

o_O => s_B_SRC_Choice

[Part 4 (c)] Draw a schematic of the simplified RISC-V processor datapath consisting only of the component described in part (a) and the register file from problem (1).



[Part 4 (d)] Include in your report waveform screenshots that demonstrate your properly functioning design. Annotate what the final register file state should be.



[Part 5 (a)] Read through the mem.vhd file, and based on your understanding of the VHDL implementation, provide a 2-3 sentence description of each of the individual ports (both generic and regular).

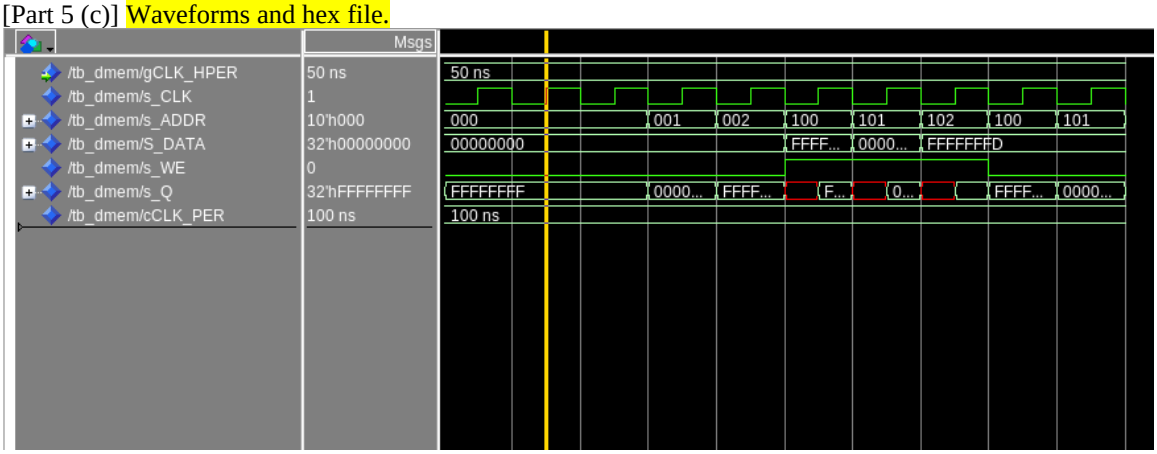
GENERIC PORTS:
DATA_WIDTH: sets a natural number (non-negative integer) size of 32 as the data width for actual memory at each data address.

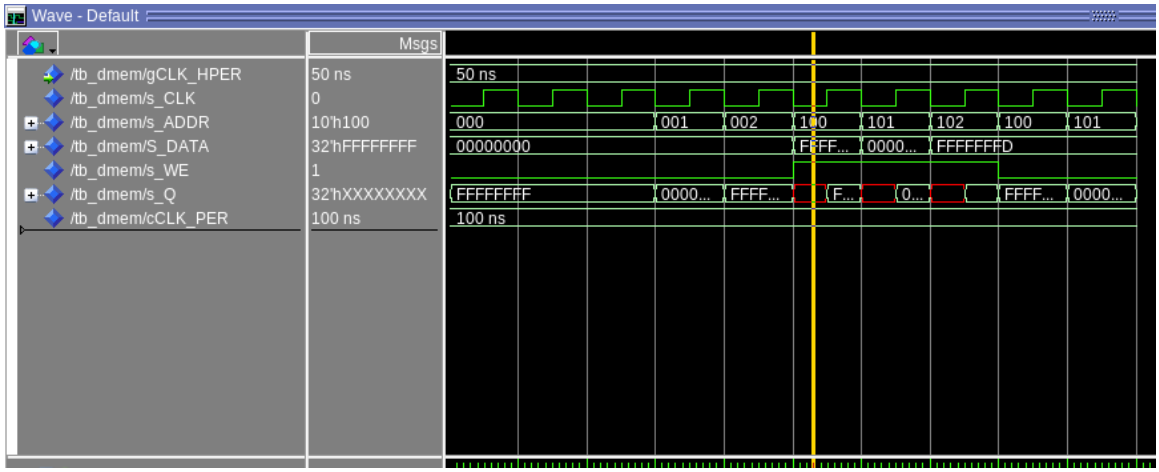
ADDR_WIDTH: sets a natural number size of 10 for addresses of where that data will actually be stored.

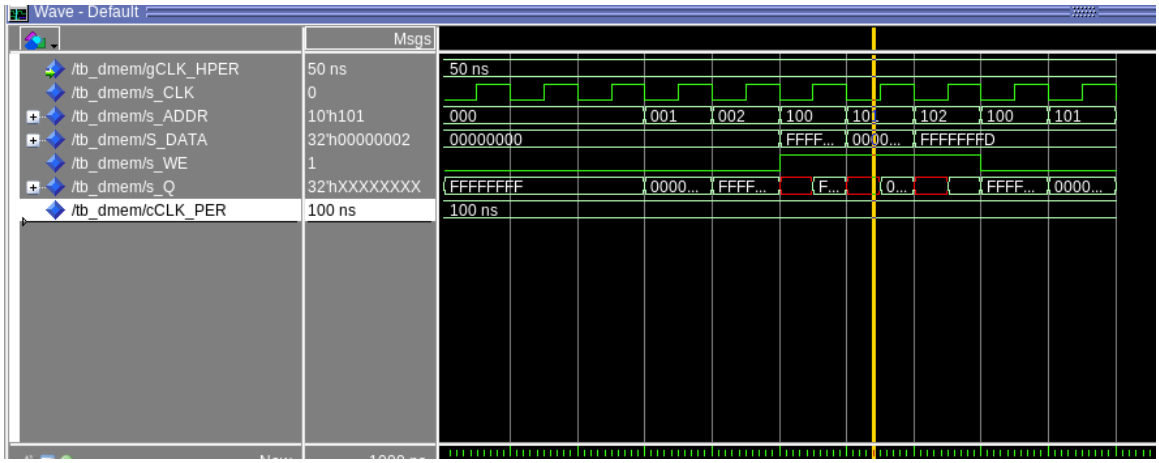
REGULAR PORTS:
clk – works as it has throughout other parts of the lab. You need a clock for synchronous single-cycle actions

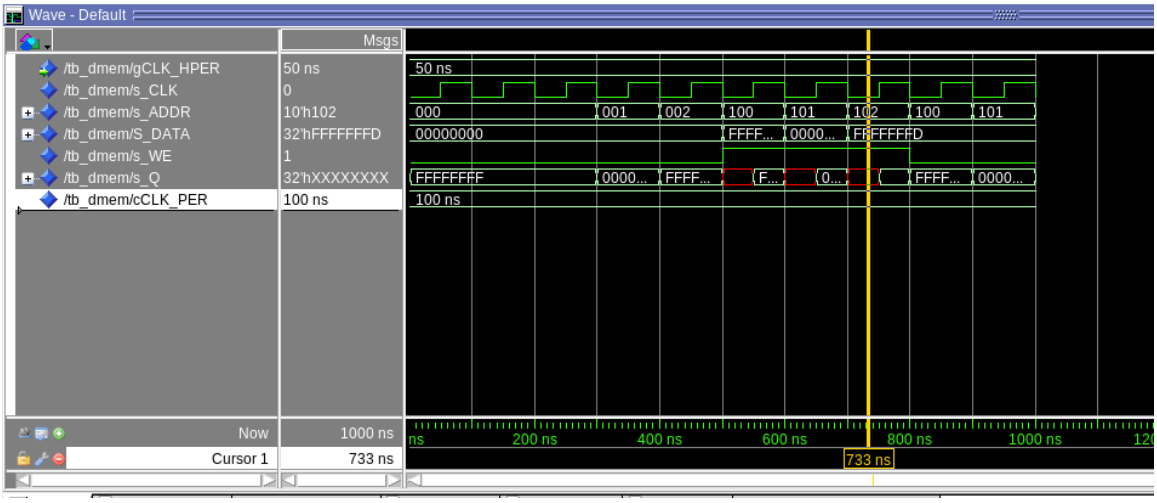
addr – an input address to where we want to write the data at
data – an input data signal (size 32-1 downto 0...so, it's 32 bits as expected) that will be the actual written information
write_enable – input control signal that determines whether we are allowed to write or not
q – looking down at the process below, if it's a rising clock edge, and if write enable is set, then ram will write "data" AT "addr". Once that process is over q will be written to with the data that is currently IN "addr"...this output will help us verify that our write was successful, it seems.

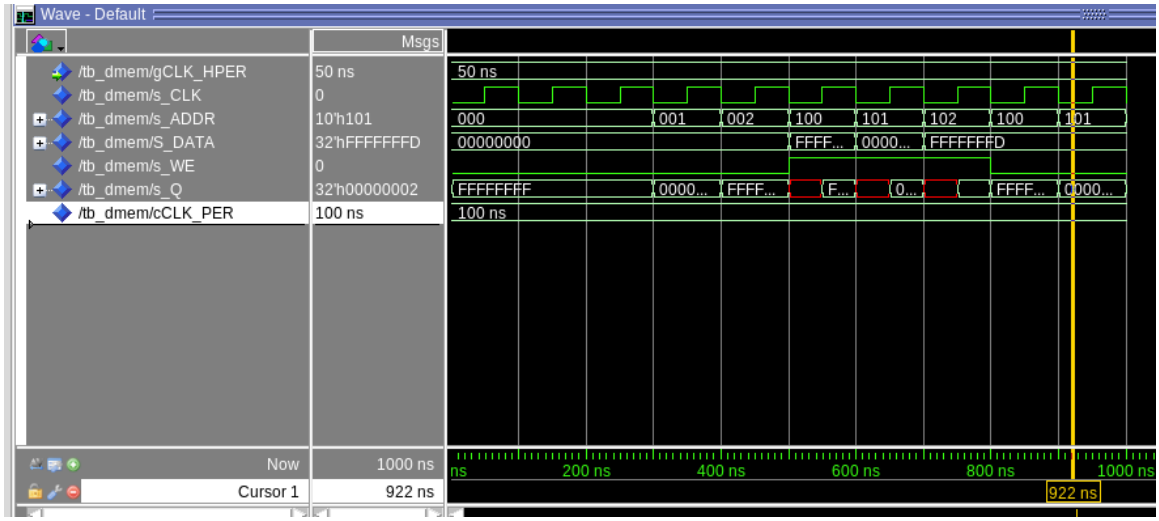
[Part 5 (c)] Waveforms and hex file.






Wave - Default
Msgs
/tb_dmem/gCLK_HPER 50 ns
/tb_dmem/s_CLK 0
/tb_dmem/s_ADDR 10'h101
/tb_dmem/S_DATA 32'h00000002
/tb_dmem/s_WE 1
/tb_dmem/s_Q 32'hXXXXXXXX
/tb_dmem/cCLK_PER 100 ns


Wave - Default
Msgs
/tb_dmem/gCLK_HPER 50 ns
/tb_dmem/s_CLK 0
/tb_dmem/s_ADDR 10'h102
/tb_dmem/S_DATA 32'hFFFFFFFD
/tb_dmem/s_WE 1
/tb_dmem/s_Q 32'hXXXXXXXX
/tb_dmem/cCLK_PER 100 ns
Now 1000 ns
Cursor 1 733 ns
200 ns
400 ns
600 ns
800 ns
1000 ns
733 ns


Wave - Default
Msgs
/tb_dmem/gCLK_HPER 50 ns
/tb_dmem/s_CLK 0
/tb_dmem/s_ADDR 10'h101
/tb_dmem/S_DATA 32'hFFFFFFFD
/tb_dmem/s_WE 0
/tb_dmem/s_Q 32'h00000002
/tb_dmem/cCLK_PER 100 ns
Now 1000 ns
Cursor 1 922 ns
200 ns
400 ns
600 ns
800 ns
1000 ns
922 ns

/home/shawnh/CPRE_381/Lab2/dmem.hex - Default

Ln# 1705 ns

```
// File Format:
//
// The text file can contain whitespace characters, comments (C-style)
// and binary ($readmemb) or hexadecimal ($readmemh) data values.  Both
// types of data values can contain x or X, z or Z, and the underscore
// character.
//
// The data values are assigned to memory words from left to right,
// beginning at start_addr or the left array bound (the default).  The
// next address to load may be specified in the file itself using @hhhh
// where h is a hexadecimal character.  Spaces between the @ and the ad
// character are not allowed.  It shall be an error if there are too ma
// words in the file or if an address is outside the bounds of the arra
//
FFFFFFFF // Loads at address 0
00000002
FFFFFFFD
00000004
00000005
00000006
FFFFFFF9
FFFFFFF8
00000009
FFFFFFF6 // Loads at address 9
```

[Part 6 (a)] What are the RISC-V instructions that require some value to be sign extended? What are the RISC-V instructions that require some value to be zero extended?

The instructions that require some value to be sign extended are…(12 bit immediate field)
addi, slti, sltiu, xori, ori, andi
lb, lh, lw, lbu, lhu,
sb, sh, sw
beq, bne, blt, bge, bltu, bgeu
jalr,
lui, auipc

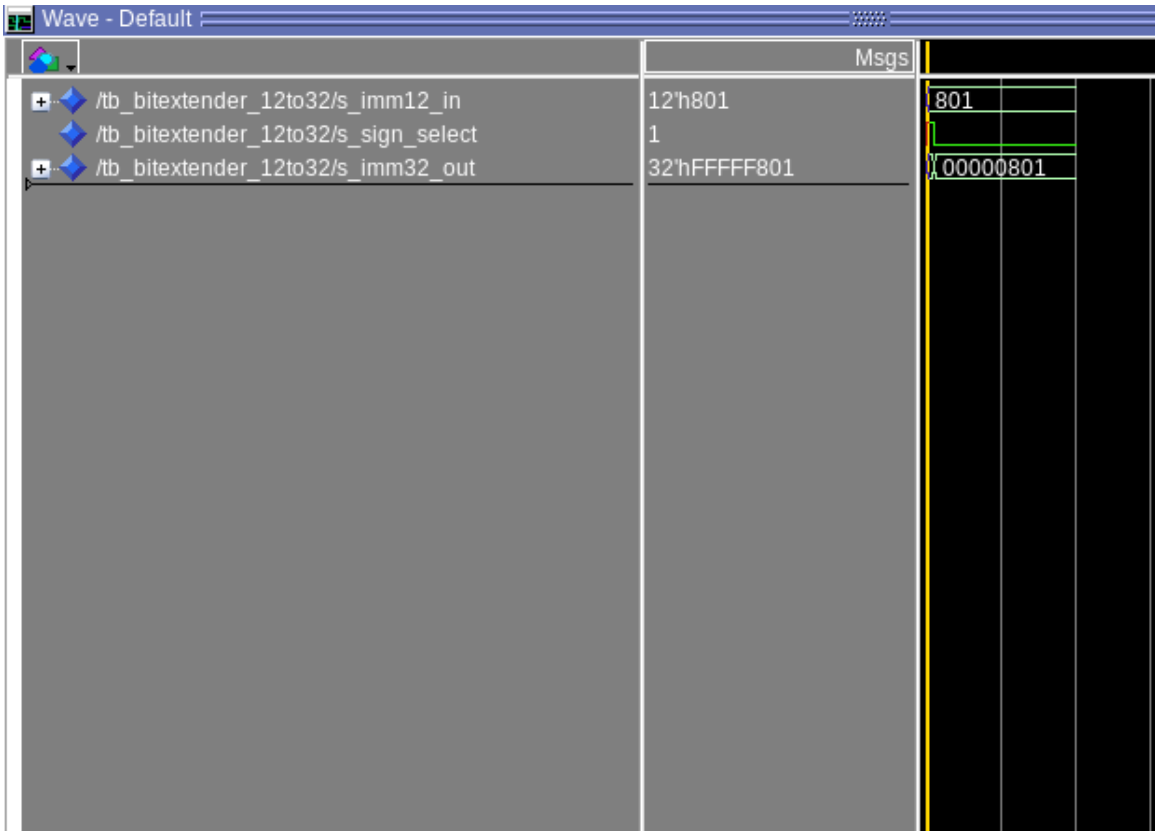
The instructions that require some value to be zero extended are…(memory data only)
lbu, lhu

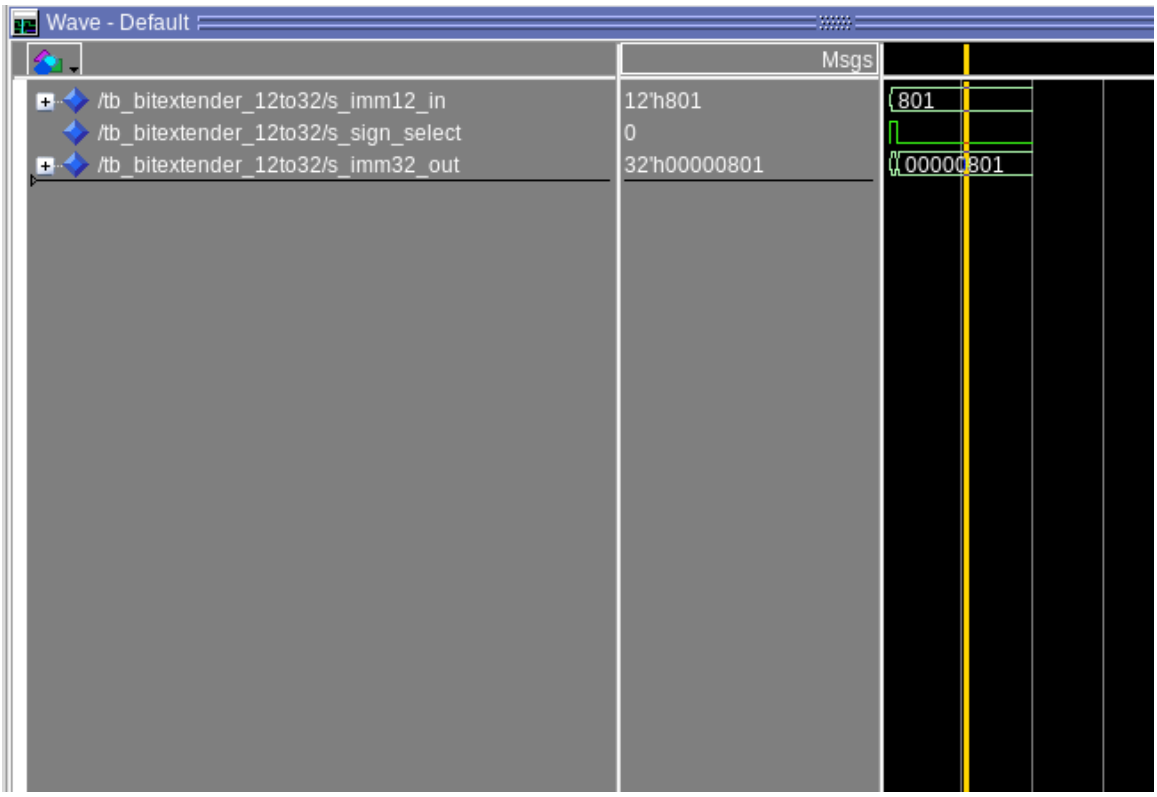
[Part 6 (b)] what are the different 16-bit to 32-bit "extender" components that would be required by a RISC-V processor implementation?

In general you would also need an 8-bit to 32-bit extender the lab document tell us, but we just need a 12-bit extender for sign and for zeroes to 32-bit

[Part 6 (d)] Waveform.

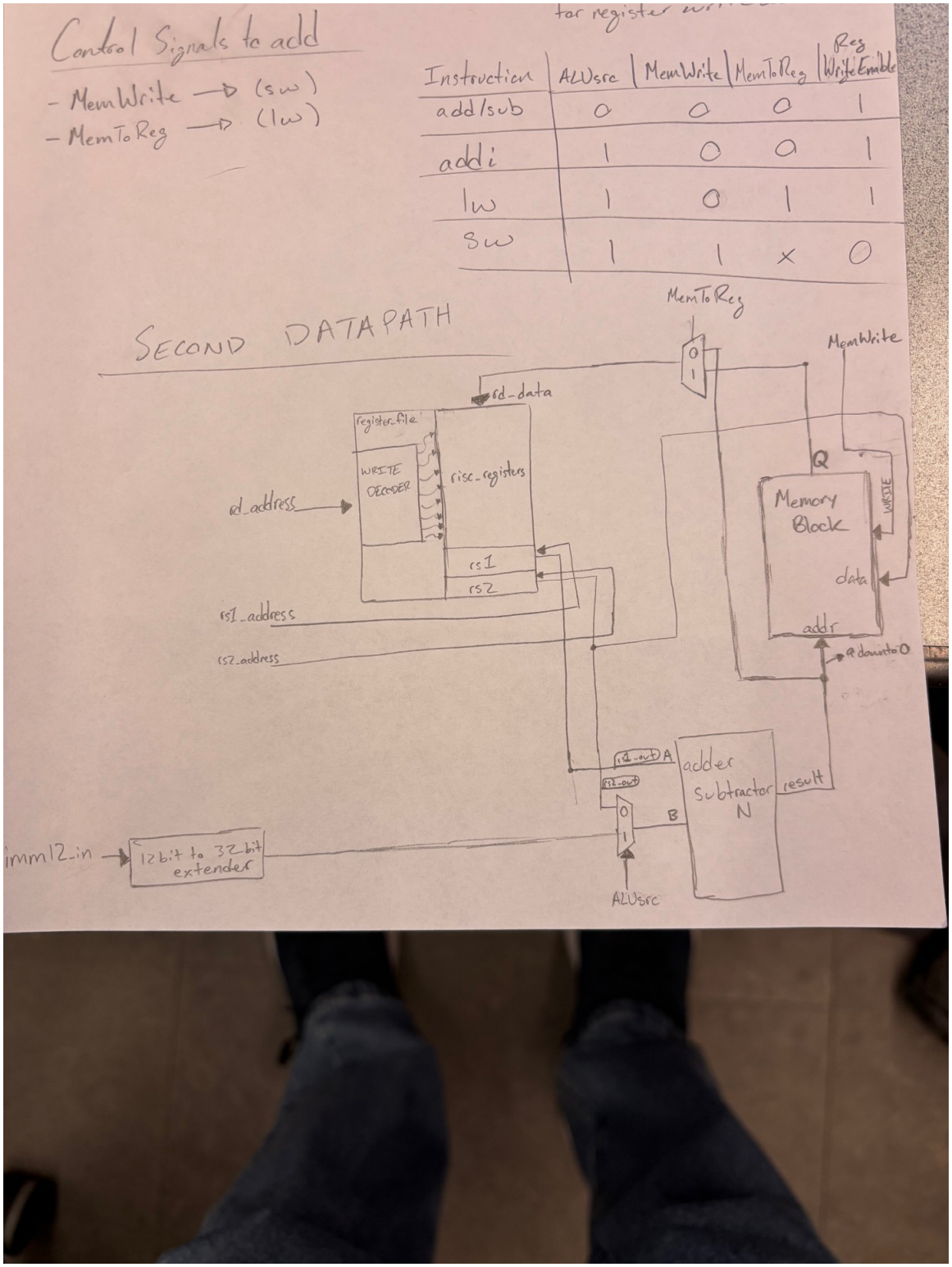
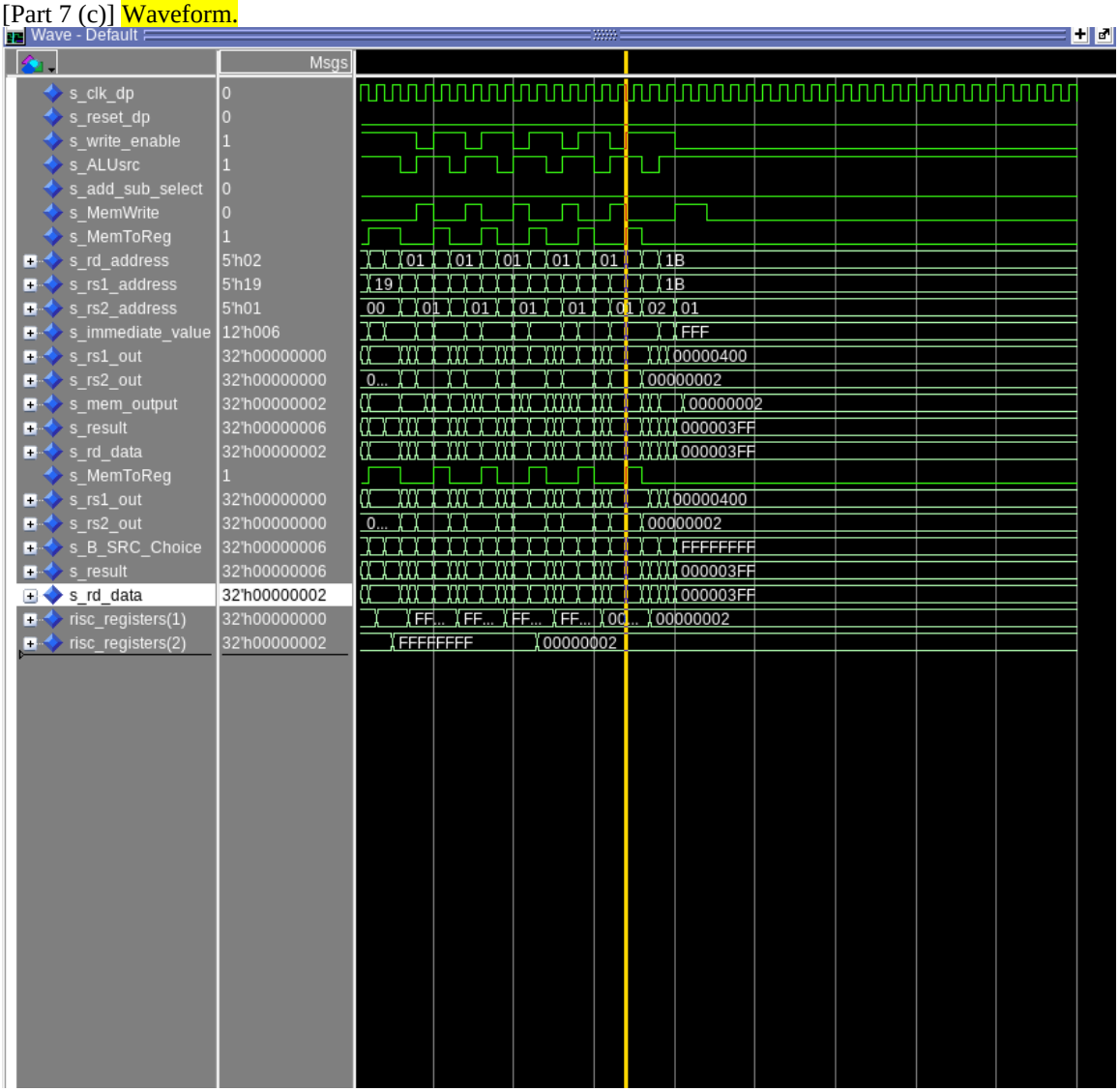
The following two pictures show sign extension working both with all F's and all zeroes


Wave - Default
Msgs
/tb_bitextender_12to32/s_imm12_in
/tb_bitextender_12to32/s_sign_select
/tb_bitextender_12to32/s_imm32_out
12'h801
1
32'hFFFFF801
801
00000801


Wave - Default
Msgs
/tb_bitextender_12to32/s_imm12_in
/tb_bitextender_12to32/s_sign_select
/tb_bitextender_12to32/s_imm32_out
12'h801
0
32'h00000801
801
00000801

[Part 7 (a)] what control signals will need to be added to the simple processor from part 2? How do these control signals correspond to the ports on the mem.vhd component analyzed in part 3?
We need to add control signals for MemToReg, and MemWrite. MemWrite corresponds to “we” on the mem entity. MemToReg coreresonds to data, as this new mux will either take in “q”, the data from the Memory Block, or it will take in my “Result” from the Adder_SubtractorN component.

[Part 7 (b)] Draw a schematic of a simplified RISC-V processor consisting only of the base components used in part 2, the extender component described in part 4, and the data memory from part 3.



[Part 7 (c)] Waveform.